

Live, Rich, and Composable: Qualities for programming beyond static text

Joshua Horowitz ¹ and Jeffrey Heer ¹

¹University of Washington, Seattle, WA

Abstract

Efforts to push beyond static textual code have worked to imbue programming with multiple distinct qualities. One which has been long-acknowledged is *liveness*: providing programmers with in-depth feedback about a program's dynamic behavior as the program is edited. A second, long-explored but unnamed until now, is *richness*: allowing programmers to work with domain-specific visualizations and interactions rather than solely through text. In this essay, we map out the relationship between these two qualities and survey past work that embodies them. We observe how systems that combine liveness and richness often do so at the cost of an essential quality of traditional programming: *composability*. We argue that programming systems embodying all three of *liveness*, *richness*, and *composability* stand the best chance of unseating static text and capturing the full potential of interactive computation. **Keywords:** Live programming, visual programming, end-user programming, composition, computational notebooks, GUIs.

1 Introduction

Alternatives to static textual code have been explored since the advent of interactive computing. Indeed, some of the earliest user interfaces ever developed were visual programming languages, using formalisms like flow charts to define programs [1], [2].

A variety of motives have driven migrations from static text. The most prominent is likely the desire to make programming more accessible. Students of programming and potential end-user programmers face serious obstacles approaching conventional programming languages. Perhaps new interfaces could make the learning process more user-friendly. Others have focused less on the temporary experience of being a newcomer and more on the way static text permanently distances programmers from their creations. (As Bret Victor memorably argued, “creators need an immediate connection to what they create” [3].) And aside from any of these user-centered motives, others still have been struck by a sense that programming interfaces ought to be capable of much more than simulating typewriters.

Driven by these motives, researchers and tool-builders have built an enormous diversity of programming systems beyond static text, ranging from spreadsheets to block-based editors to exotic visual programming languages. It is clear that static text is a single point in vast, multidimensional space of possible programming environments. As an amorphous cloud of case-studies spanning decades, this design space is overwhelming. To make sense of it, and to understand the progress that has occurred, we propose identifying fundamental qualities that these designs advance beyond static text.

In this essay, we focus on two qualities that have emerged as fundamental to efforts beyond static text.

The first quality is *liveness*: providing programmers with in-depth feedback about a program's dynamic behavior as the program is edited. This quality is well-established in the computer-science literature.¹

The second quality is as yet unnamed, though its pursuit is implicit in many projects: allowing programmers to work with domain-specific visualizations and interactions. We offer *richness* as a new term of art to better pin down this quality for analysis.

In this article, we will look at these two qualities, surveying systems that exemplify them in different combinations. In the course of this analysis, we will find it necessary to invoke a third quality called *composability*: the ability to freely combine smaller programmed artifacts into larger ones, to accomplish larger goals. Unlike liveness and richness, this is not a quality static text lacks, which new programming systems strive to add to it. Rather, it is a familiar quality of static text which new programming systems will have to work hard to maintain.

¹Liveness was named as early as 1990 [4], and a workshop named after it has been running since 2013 [5].

PLATEAU

13th Annual Workshop at the
Intersection of PL and HCI

DOI: 10.35699/1983-
3652.yyyy.nnnnn

Organizers:
Sarah Chasins, Elena
Glassman, and Joshua
Sunshine

This work is licensed under a
Creative Commons Attribution 1
4.0 International License.

By combining *liveness*, *richness*, and *composability*, programming systems can go meaningfully beyond the capabilities of static text, without losing its unique power. We hope that our survey and analysis helps to further define this space. [TK: fix this concluding graf]

2 Liveness

Programming is conventionally an opaque art. As a computer runs a program, it steers itself through control structures, performing operations and constructing intermediate values. Programmers need to carefully orchestrate this behavior but, most often, the behavior is invisible. No feedback is provided by the computer to the programmer to tell them what their program will do as they write it. Instead, programmers are tasked with simulating in their heads how their program will run without computational assistance. While debuggers can provide some of this information, they do so through a narrow peephole intended for extraordinary (“debugging”) situations. While editing code, programmers still have no visibility into the effects of their edits on run-time behavior.

Live programming refers to efforts to move away from this status quo, creating “programming tools which provide immediate feedback on the dynamic behavior of a program even while programming” [6]. The use of “live” in this context dates to Tanimoto [4], although systems embodying various levels of liveness date back as far as computing itself.

Why would a programmer want liveness in their programming system? A survey of work in live programming identified 187 papers in the field and identified ten motivations these papers give for liveness [6]. They include improved productivity for programmers, often by speeding up programming tasks, increased accessibility for less experienced programmers, and eased comprehension while reading existing code.

Before digging into some specific examples of live programming systems, a quick note on what we do not mean by “live”: First, confusingly, the term “live programming” (which we are discussing) has taken on a separate meaning from “live coding”. The latter refers to artists editing code as part of a performance, most commonly to create music or video art. While this practice often relies on programming tools that are “live” in the “live programming” sense, artistic performance encompasses only a small fraction of the use-cases that live programming is meant to support. (See Rein et al. [6] for more on this distinction.) Also, we do not use “liveness” to refer to interfaces surfacing information about a program being edited using only static-analysis techniques, such as type-driven autocomplete or warning messages. If an interface can be used without the underlying program being run, it does not embody “liveness”.

The definition of liveness we have chosen here matches some definitions given in the literature (such as in Rein et al. [6]). At the same time, we hardly believe there is an established consensus on what the most useful precise definition of liveness is. We invite further discussion on this question, and hope that our analysis here is helpful even to those who prefer a different definition. [TK: is this cringe? should this go in a paper conclusion?]

2.1 Approaches to liveness

Developers of live programming systems have experimented widely with the interfaces and mechanisms by which liveness can be achieved. Here, we quickly classify a number of examples into a few imprecise but helpful buckets based on the general approaches taken.

Liveness outside of code. Some systems provide quick feedback about the final output of a program, without revealing information on program internals that led to that output. Examples include “live reloading” and “hot reloading” systems popular for application development [7] and split-screen editors popular for generative art (the P5.js web editor [8], the tree & video-game examples from Victor [3], and TouchDevelop as described in Burckhardt et al. [9]). These often require special technical work to update quickly or to preserve run-time state during updates. However, as we discuss in the “Structural granularity” section of §2.2, below, this is structurally the coarsest liveness possible, as the other examples of liveness we describe below all provide visibility to the internals of programs, rather than just to their outputs.

Liveness within textual code. One common recipe is to augment textual-code editors with

in-context displays showing run-time behavior. Rauch et al.'s "Babylonian-style Programming" [10] surveys the state of the art in this category as of 2012. They evaluate eight existing editors [3], [11]–[16], before adding their own to the list. Notable entrants to this category since 2012 include Swift Playgrounds in Xcode [17] and Projection Boxes [18].

Liveness between textual code cells. Rather than threading live feedback into traditional codebases, other editors provide special interfaces where code is broken up into "cells". These editors can then provide visibility into values flowing between cells. Recent cell-based live systems include Observable [19] and Natto [20], which both augment JavaScript programming in-the-small with a live structure in-the-large: Observable via a computational notebook and Natto via nodes and wires on a canvas.

Liveness with structure editors. *Structure editors* (sometimes called *projectional editors*) are not based on textual code or cells thereof, but instead allow interactions directly with structured representations of code. Structure editors are constructed from scratch as entirely custom interfaces, so they present an opportunity to deeply intertwine code and live feedback. Many projects take advantage of this opportunity, including Enso [21], Lamdu [22], PANE [23], and Subtext [24].

Liveness in a dynamic environment. Some early live programming environments, including Smalltalk [25], Lisp OSes like Genera [26], and Boxer [27], do not fit neatly into the above categories. These are environments where the line between code and application is not so clearly drawn. While these environments do not provide live visibility into running code, running blocks of code produce immediate, visible effects in these environments, forming an important part of their use.

2.2 Levels of liveness

Liveness is not a yes-or-no proposition. Rather, it is a quality which can be embodied to greater or lesser degrees along different axes.

Structural granularity. How detailed is the feedback provided by a live system? At one end of this spectrum lie systems that display only feedback about a program as a whole, which we discussed under the heading "Liveness outside of code" above. While fast feedback of this form is an improvement on slow compile-run loops, the internals of a program (intermediate values and control flow) remain invisible. Increasing granularity from here, we reach systems that provide live feedback on a "cellular" level, as described under the heading "Liveness between textual code cells" above. By breaking up a larger program into cells, programmers can receive feedback below the whole-program level. Finally, we reach the most fine-grained end of the spectrum, where feedback is threaded through constructs of the programming language itself. A finer structural grain would seem to be advantageous, in providing the most feedback to the programmer. However, excessive feedback can be distracting and can result in low information density, diluting code with irrelevant feedback. In situations where the programmer does not explicitly signal to the system where feedback should be provided (by providing a division of code into cells, for instance), careful visual and interaction design is needed to ensure feedback doesn't get in the way.

Temporal granularity. How often are edits responded to with feedback in a live system? As an example of moving along this spectrum, consider Observable. Observable is a reactive notebook, which re-runs a cell reactively whenever its code or an upstream dependency value changes. In this way, it is more live (its liveness has a finer temporal grain) than Jupyter, which requires cells to be re-run manually, even if their dependencies change. [TK: citations? more description?] However, a code change in Observable only takes effect when it is saved manually or its text editor loses focus, not on every keystroke. In this way, it is less live (its liveness has a coarser temporal grain) than Natto, which can re-run cells on every keystroke. A finer temporal grain would seem to be advantageous, in providing feedback to the programmer about the effects of their actions as quickly as possible. In this way, programming can begin to feel more like aiming a water hose than aiming a bow-and-arrow, to use a striking metaphor from Hancock [28]. However, it can also be dangerous. Code usually moves through invalid states on the way from one valid state to another. If these invalid programs are indiscriminately executed, this can make the programmer's experience sluggish, produce meaningless visual output, and possibly even trigger damaging side effects. (Some cell-based systems, including Natto and Hex, allow

the user to select how often each cell should be re-evaluated, thereby allowing the user to customize temporal granularity on a cell-by-cell basis.)

Tanimoto's levels. In his paper coining “liveness”, Tanimoto proposed his own four-level hierarchy of liveness [4]. In a follow-up paper, he extended this with two additional levels [29]. In Tanimoto's taxonomy, liveness takes on a broader definition than we are using here. His levels 1 & 2 do not require any liveness at all, in our sense of the word, and instead focus on the representation of code itself as a visual structure. His levels 3 & 4 match our sense of liveness more closely, with the distinction between 3 and 4 arguably coming down to a difference of temporal granularity. His levels 5 & 6 once again diverge from our definition of liveness, as they refer to the programming system anticipating the programmer's future actions and synthesizing new code automatically. While this could, with a stretch, be deemed a form of “feedback”, it goes beyond the goal of passive visibility that we intend to capture by “liveness”.

3 Richness

People use a vast range of rich, specialized computer interfaces in daily life. We use spreadsheets to track accounts, graphics editors to create images, and timeline views for video editing. These interfaces are crafted around their domains, offering representations that afford direct manipulation and visual feedback.

Turning our gaze to static-text programming, this richness disappears. Static text does not afford heterogeneous, domain-specific representations. Parsing input, processing images, and building a visualization are all distinct tasks, but a programmer's screen looks the same no matter which they are working on. “Domain-specific” languages can offer separate dialects for these separate situations, but even with DSLs, programming still takes on a textual form, deprived of the direct manipulation and visual representations that characterize everyday computer use.

Many efforts, from academia and industry, have explored bringing this sort of domain-specific richness to programming. Yet there is, to our knowledge, no succinct term analogous to “liveness” that can be used to describe it. We propose “richness” as a name for this quality. A programming system embodies “richness” insofar as it allows programmers to work with visualizations and interactions that are tailor-made to the tasks, domains, and contexts they work in. Like liveness, richness is meant to be a term of art: just because something feels “rich” in a colloquial sense doesn't mean it will be rich in our specific sense.

Why would a programmer want richness in their programming system? We believe many of the motivations for liveness apply here as well, including improving productivity, accessibility, and comprehension of programming systems.

But a more specific argument can be made for richness, focused on the way visual representations allow entirely new forms of communication and thought. Andersen et al. [30] write:

[W]riting code means articulating thoughts as precisely as possible... Often these thoughts involve geometrical relationships: tables, nests of objects, graphs, etc. Furthermore, the geometry differs from problem domain to problem domain. To this day, though, programmers articulate their thoughts as linear text. Unsurprisingly, code maintainers have a hard time reconstructing geometrical thoughts from linear text, which reduces their productivity.

Similarly, Omar et al. [31] write:

Diagrams have played a pivotal role in mathematical thought since antiquity, indeed predating symbolic mathematics. Popular computing and creative tooling, too, has embraced visual representation and direct manipulation interfaces for decades. Programming, however, has remained stubbornly mired in textual user interfaces.

Before digging into some specific examples of rich programming systems, a quick note on what we do not mean by “rich”: A programming system being “visual” does not, in and of itself, make it

rich in our sense. Many visual programming systems use visual structures to represent the general-purpose, generic structures of textual programming, such as dataflow or control structures. This is not heterogeneous, domain-specific richness.² Programming systems that are visual without being rich include:

- block-based programming languages like Scratch [32],
- node-and-wire editors like Natto [20] and Unreal Engine's "Blueprints" [33],
- visual structure editors like the "frame-based" editor in Greenfoot [34], or the work described in Read & Marlin [35],
- classic "visual programming languages", including four out of five of those surveyed in Boshernitsan et al. [36]: VIPR [37], Prograph [38], Forms/3 [39], and Cube [40].

Our definition of richness is early and speculative, and we do not expect complete agreement on its delineation. For instance, some may disagree with our choice to focus on heterogeneous, domain-specific UIs, excluding the general-purpose visualizations listed above. We welcome continued conversation as part of the shared process of sense-making in our community. [TK: is this cringe? should this go in a paper conclusion?]

3.1 Liveness without richness, richness without liveness

Although richness and liveness can be deeply intertwined, they are still distinct qualities. We can make this clear by examining systems that embody liveness without richness or visa versa.

Liveness without richness. All the examples we presented in §2 were intentionally chosen to lack richness. This includes, for instance, text-based programming environments like those described in "Babylonian-style Programming" [10]. Though these environments provide visibility into what code is doing, the code itself is still edited as generic static text, without any domain-specific visual elements.

Richness without liveness. In a programming system that is rich but not live, richness often takes the form of "visual syntax" – directly editable visual representations which act as "syntactic sugar" over traditional code. An early example is Erwig & Meyer [41], which shows examples of embedding state diagrams, AVL trees, and pointer-based data structure diagrams into code. Andersen et al. [30] describes a visual macro system for Racket which lets users define interactively editable visual syntaxes embedded inside of arbitrary Racket code. They also demonstrate this with tree diagrams, as well as puzzle games and inline forms. Graphite extends a Java IDE with pop-up "palettes" to bidirectionally edit literals (like colors, regular expressions, and inline SQL) in code [42]. MPS (Meta Programming System) is a commercial workbench for developing projectional domain-specific languages. MPS languages typically look mostly like conventional code, but they can embed custom interfaces like tables and state-machine diagrams [43]. Arawjo et al. [44] explores integrating hand-drawn quantum-circuit diagrams into textual code in Jupyter notebooks, an example of what the authors term "notational programming". Although all these examples add richness to static text, they remain static – they do not respond to dynamic activity of a running program with live feedback.

3.2 Richness with liveness

We now turn to the intersection of liveness and richness, a combination which opens up synergetic possibilities.

Chief among them is the possibility of *programming by demonstration* (PbD) [45]. In a PbD system, a user interacts with concrete objects via direct manipulation. The system is able to generalize beyond this specific "demonstration" to new inputs, turning the demonstration into a reusable program. Projects have explored how to achieve this goal in different domains, from wrangling tabular data [46] to building custom visualizations [47]–[50] to building entire web applications [51]. [TK: am I going through these too fast?] Naturally, the mechanisms they use vary widely. However, all PbD systems must be *live* and *rich*. They must be *live*, because a demonstration is performed on concrete

² Andersen et al. [30] articulates this limitation: "[visual languages] offer only a fixed set of constructs, though visual ones—meaning a visual language fails to address the problem-specific nature of geometric thought." This echoes Victor [13], which writes: "Traditional visual environments visualize the code. They visualize static structure. But that's not what we need to understand. We need to understand what the code is *doing*. Visualize data, not code. Dynamic behavior, not static structure."

input data, and it must change as the user acts on it. This data is only available at run-time, so a purely static system cannot support PbD. And they must be *rich*, because direct manipulation requires domain-specific representations and interactions.

Programming by demonstration is one particularly exciting category of live & rich tools. But it is not the only approach live & rich tools take. Some still require the user to express their intent symbolically, but use liveness and richness to make this process easier. Examples like this include InterState [52], Apparatus [53], Object-Oriented Drawing [54], and various regular-expression development tools [55]–[57].

4 Composability

A pattern is implicit in our previous discussions. When discussing “liveness without richness” and “richness without liveness” (in §3.1), most example systems positioned themselves as general-purpose, roughly the analogue of a conventional general-purpose programming language like Python.

Not so, in our more recent discussion of “richness with liveness” (in §3.2). The tools described in that section were targeted at specific domains. Indeed, their strength came largely from this specificity. Each of these tools replaced generic symbolic programming processes with direct interactions tailored specifically to their domain.

However, this leaves an important question on the table: How can these powerful but specific tools be combined together to solve real-world programming goals? Real-world programming often requires performing multiple tasks in concert. For instance, suppose a data analyst wants to understand how well a product launch is proceeding. They may need to:

1. acquire data from an API
2. reformat this data from semi-structured JSON into a tabular format
3. wrangle the tabular data further
4. build charts to show patterns in this data.

Each of these steps could be supported by a targeted live and rich tool, akin to the ones we discussed in “richness with liveness”. In what sort of environment could the analyst fluidly make use of these tools *together*?

What we are asking for here is *composability*: the ability to freely combine smaller programmed artifacts into larger ones, to accomplish larger goals. Composability is a pervasive quality of conventional programming, visible whenever we write Python scripts that call out to multiple libraries, or even simply when we nest `if` and `for` inside one another. Ultimately, this ability to freely combine building blocks is what gives programming its unlimited expressive potential.

The most important question this paper asks is: How might programming acquire the desirable qualities of *liveness* and *richness* without giving up the crucial quality of *composability*?

We will first discuss a few current approaches which we believe do *not* provide real composability, before surveying a few systems, some very recent, which do aim to combine all three qualities from the start.

4.1 Current approach: Manual integration

The live, rich, domain-specific tools we discussed in §3.2 usually provide *some* forms of integration with the outside world, if only for demonstration. This may take the form of copying and pasting text snippets, like copying regular expressions in and out of a regular-expression development tool, or copying generated code out of (and back into) Wrangler [46]. Alternatively, it may take the form of importing and exporting files, like how tabular data is loaded into and out of Wrangler. In any case, we call this *manual* integration, as it requires the user propagating changes between the tool and its surroundings by hand.

Two kinds of integration are often essential. First of all, if a tool is a *programming* tool, that means that it provides some sort of re-usable behavior. To incorporate this into a larger project manually, the tool must produce some sort of code. (“Code” here is meant broadly, and can include such executable notations as regular expressions or JSON-encoded visualization specifications [58].) Second of all, if a live, rich tool is used to define behavior which takes in input data (such as a regular expression taking

in a string to match on, or a chart taking in data to plot), then the tool itself will real instances of such data, to drive a live, rich programming experience. A data analyst using a chart-builder tool will want to build their chart in the presence of real data!

[TK: I dunno how to fit these all together yet]

On some level, manual integration works, and can provide the above forms of integration. A certain workflow is possible, copying program snippets and data back and forth between a tool and a codebase. However, this workflow leaves much to be desired.

First, most obviously, there is a lack of *collocation*. Using a separate tool requires leaving a programmer's preferred environment and entering a new space. With this comes a potential for disorientation and distraction.

Second, there is a lack of *full persistence*. For instance, although a programmer may curate a set of test inputs to a regular-expression helper tool as they use it, this ensemble is not persisted when the regular expression is copy-pasted back to code.

But most importantly, there is a lack of *liveness*. If a situation requires the use of multiple composed tools, work in one tool may produce downstream effects that affect another tool. For the overall workflow to be live, in a holistic sense, these effects should be visible as quickly as possible, with as little distracting effort as possible. However, in a manually-integrated system, the programmer must propagate changes to a tool's inputs and outputs by hand. Although the tool may be live *internally*, there is no liveness across the boundaries between tools. This weakens the benefits of tools' internal liveness.

These problems add up to a significant cost when integrating tools manually. Because this cost occurs on a per-tool basis, it disincentivizes the development of small tools, even though, integration cost aside, ensembles of small tools would allow for greater flexibility and combinatorial possibility. In an environment with high integration costs, only large, monolithic tools will survive.

4.2 Current approach: Bundles

If manual integration carries heavy costs, a natural alternative is to combine together, in a single tool, multiple live & rich tools a user would want to use for a particular larger-scale task.

A good example of this approach can be found in Gneiss [51]. Gneiss is a tool for making interactive, data-driven web applications. It combines, in three panes, three separate tools: an API query tool, a spreadsheet, and an interface builder. These tools support a fluid workflow where the user crafts an API call, drags pieces of the response into the spreadsheet, and builds an interface off of this spreadsheet. Data flows between these tools automatically, allowing an integrated live experience that would not be possible with tools that were manually integrated into a static codebase. Gneiss makes a compelling demonstration of the benefits of integrated liveness – liveness not just within tools, but between them.

However, Gneiss makes this integrated liveness available only for one particular, hard-wired bundle of tools. This means that, if a user's workflow is close to, but not quite, the workflow anticipated by Gneiss's creators, Gneiss will fail to meet their needs. What if a user wants to build an interface off of imported CSV data, rather than JSON from an API? What if a user wants to construct a bespoke chart from their data, rather than a web app? Even if tools exist to support these steps, a Gneiss user cannot incorporate them.

A similar limitation holds in the other direction. To build Gneiss, its creators put significant work into designing and implementing three ingenious live tools. The domains of these tools presumably extend further than the specific workflow that Gneiss was designed around. Since these tools are built into Gneiss, however, they cannot be brought into new environments or contexts.

If we judge Gneiss by the standards of modern application software, these limitations are not surprising. We have not come to expect open-ended reconfigurability and modularity in the world of graphical software interfaces.³ From the perspective of programming systems, however, these

³ A partial exception to this is applications supporting plugin or extension mechanisms, such as web browsers and sophisticated creative tools. However, the point still stands that typical computer use is not characterized by fluid recombination of modular interface elements.

limitations are striking. If Gneiss's capabilities were represented with traditional code, they would look like a fixed script, making use of three specific libraries connected in one particular way. A user's interactions with Gneiss would modify blank slots within this structure. However, the user cannot import additional libraries, or rearrange the existing libraries into a new structure. The open-ended composability that is present in conventional programming is absent in the world of graphical interfaces, even live and rich interfaces like Gneiss.

4.3 Liveness, richness, and composability

We have seen that attempts to compose together tools *manually* fail to preserve liveness across tool boundaries, and attempts to compose together tools into pre-made *bundles* fail to provide users with open-ended composability. For a system to overcome these limitations, it would need to provide open-ended composition of rich interfaces into an integrated live system.

Several recent projects aim to do just this.

mage. mage is “an API for allowing smooth transitions between GUI and code work in notebooks”, implemented as a Jupyter Notebook extension [59]. Tools made with mage can be launched from cells of Jupyter Notebooks. A tool is rendered with its cell, and it can read from and write to the code in that cell. mage is demonstrated with a variety of tools, including a chart builder, an image editor, and a table editor which converts direct-manipulation actions into dataframe manipulation code.

Liveness in mage comes from the way that interacting with a tool causes its cell's code to be rewritten and rerun automatically, and from the way that cells with tools can be quickly re-run to reflect any changes in their inputs. (This liveness is limited a bit by the non-reactive nature of Jupyter notebooks. As we put it earlier, liveness in Jupyter has a coarse temporal grain. However, this is inessential to mage's approach, and it is easy to imagine mage being implemented on top of a reactive notebook instead.)

Richness in mage comes from its support for tools with open-ended visualizations and interactions, using any features available on the web platform.

Composability in mage comes from its Jupyter notebook host. Multiple tools can be used in concert by placing them into separate cells, feeding data between them with variables.

[TK: limitations... notebook is flat, tools can only be composed through separate cells.]

Livelits. Livelits (live literals) are an extension to Hazel [60], “a live functional programming environment designed around hole-driven development” [31]. Hazel itself is a live structured editor which emulates conventional statically-typed functional code. Livelits add “user-defined GUIs embedded persistently into code” [31] to this. Livelits are demonstrated with a few such GUIs, including sliders, color pickers, table literals, and image editors.

Liveness in Livelits is derived from the liveness of the Hazel platform that underlies it. Hazel's liveness is fine-grained both structurally and temporally, and these qualities extend to Livelits.

Richness in Livelits comes from [TK]

Composability in Livelits is also derived from the liveness of the Hazel platform. [TK]

[TK: limitations... livelit must specify an expansion type, further from practice than mage]

4.4 Current approach: Other notebooks

[TK: not sure where to put this, or whether it's important enough to include]

At first glance, Observable may seem to embody liveness and richness together with composability. Although it is indeed close to this goal, it fails to achieve it. The most important limitation standing in the way of this is that, although Observable cells can render UI, this UI cannot persistently modify program state. That is, even though an Observable cell can render an arbitrarily rich tool as part of its output, interactions with this tool cannot be “saved” back to the notebook, and they will always disappear when the notebook is reloaded. If programming tools are to be part of the process of writing programs, rather than just ephemeral use thereof, it is essential that persistence from rich UIs back to a source of program truth be possible.

The appeal of live, rich (& persistent) tools is strong enough that several platforms, including Observable, have added them as one-off features. Observable has a “Data Table Cell” feature, which

lets the user embed a live, rich direct-manipulation table editor in place of an ordinary cell [61]. However, this is built into the Observable platform and can access capabilities that are not available to user-authored tools. Similarly, the commercial notebook provider Hex offers “no-code” cells: live, rich tools for building charts, pivoting and filtering data, and other specific tasks [62]. These are similarly built-in.

One could imagine Observable or Hex offering an API by which cells could spawn editors that can persist state. There are, of course, many details to work out, such as supporting synchronous multi-user editing. It may be too much to expect a commercial venture such as Observable to go out on a limb to explore these ideas. Clearly, the vision of composable programming we are trying to develop here cannot be satisfied [TK] with a closed world of.

5 Conclusion

In this paper, we defined *liveness* and *richness*. We explored examples that embody them, alone and in combination. And we proposed that, for them to be combined together in ways that support real use, they must maintain the composability that makes conventional programming so versatile and adaptable.

The design space at the triple intersection of liveness, richness, and composability has just begun to be explored through recent projects like mage and Livelits. Above all else, we hope that our work to map it out will encourage further ventures into this space.

Some specific questions spring to mind:

Composing interfaces. Along the way to making live, rich interfaces composable, a system designer must make interfaces (at least certain ones) composable. This is not a straightforward task. Modern computing platforms do not support fine-grained composition of interfaces by users. Efforts in the past to accommodate such composition, like OpenDoc [63], have failed. mage and Livelits both find some preliminary success leveraging the Web platform for composition of interfaces. Continued development in practice will require resolving numerous difficulties, from encapsulating CSS layout to sand-boxing for security. Pre-existing component systems for the web, such as Web Components [64], React [65], and the Block Protocol [66] may help here, though none perfectly addresses the needs of live programming tools.

[TK: more questions? or don't include questions at all?]

[TK: conclusion?]

References

- [1] T. Ellis, J. Heafner, and W. Sibley, *The GRAIL Language and Operations*. RAND Corporation, 1969.
- [2] T. O. Ellis, J. F. Heafner, and W. L. Sibley, “The GRAIL PROJECT: An EXPERIMENT IN MAN-MACHINE COMMUNICATIONS,” 1969.
- [3] B. Victor, “Inventing on Principle,” <https://vimeo.com/36579366>, Presented at the the Canadian University Software Engineering Conference (CUSEC), 2012.
- [4] S. L. Tanimoto, “Viva: A visual language for image processing,” *Journal of Visual Languages and Computing*, vol. 1, no. 2, pp. 127–139, Jun. 1990.
- [5] *LIVE 2013: Workshop on Live Programming*. <http://liveprogramming.github.io/2013/>.
- [6] P. Rein, S. Ramson, J. Lincke, R. Hirschfeld, and T. Pape, “Exploratory and Live, Programming and Coding,” *The Art, Science, and Engineering of Programming*, vol. 3, no. 1, Jul. 2018.
- [7] D. Abramov, *Live React: Hot Reloading with Time Travel*. Jul. 2015.
- [8] p5.js, *p5.js Web Editor*. <https://editor.p5js.org/>.
- [9] S. Burckhardt, M. Fahndrich, P. de Halleux, *et al.*, “It’s alive! continuous feedback in UI programming,” *ACM SIGPLAN Notices*, vol. 48, no. 6, pp. 95–104, Jun. 2013.
- [10] D. Rauch, P. Rein, S. Ramson, J. Lincke, and R. Hirschfeld, “Babylonian-style Programming: Design and Implementation of an Integration of Live Examples into General-purpose Source Code,” *The Art, Science, and Engineering of Programming*, vol. 3, no. 3, Feb. 2019.

- [11] J. Edwards, "Example centric programming," *ACM SIGPLAN Notices*, vol. 39, no. 12, pp. 84–91, Dec. 2004.
- [12] T. Imai, H. Masuhara, and T. Aotani, "Shiranui: A live programming with support for unit testing," in *Companion Proceedings of the 2015 ACM SIGPLAN International Conference on Systems, Programming, Languages and Applications: Software for Humanity*, ACM, Oct. 2015.
- [13] B. Victor, *Learnable Programming*, <http://worrydream.com/LearnableProgramming/>, 2012.
- [14] C. Granger, *Light Table*, <http://lighttable.com/>, 2022.
- [15] T. van der Storm and F. Hermans, "Live Literals," <https://homepages.cwi.nl/~storm/livelit/livelit.html>, Presented at the Workshop on Live Programming (LIVE) 2016, 2016.
- [16] S. Kasibatla and A. Warth, "Seymour: Live Programming for the Classroom," <https://harc.github.io/seymour-live2017/>, Presented at the Workshop on Live Programming (LIVE) 2017, 2017.
- [17] Apple Inc., *Wwdc 2014*, <https://www.youtube.com/watch?v=w87fOAG8fjk>, 2014.
- [18] S. Lerner, "Projection Boxes: On-the-fly Reconfigurable Visualization for Live Programming," in *Proceedings of the 2020 CHI Conference on Human Factors in Computing Systems*, ACM, Apr. 2020.
- [19] Observable Inc., *Observable - Explore, analyze, and explain data. As a team.* <https://observablehq.com/>, 2022.
- [20] P. Shen, *Welcome! – natto*, <https://natto.dev/>, 2022.
- [21] Enso, *Enso*, <https://enso.org/>, 2022.
- [22] E. Lotem and Y. Chuchem, *Lamdu*, <https://www.lamdu.org/>, 2022.
- [23] J. Horowitz, "Pane: Programming with Visible Data," <http://joshuahhh.com/projects/pane/>, Presented at the Workshop on Live Programming (LIVE) 2018, 2018.
- [24] J. Edwards, "Subtext," in *Proceedings of the 20th annual ACM SIGPLAN conference on Object oriented programming systems languages and applications - OOPSLA '05*, ACM Press, 2005.
- [25] A. Goldberg and D. Robson, *Smalltalk-80: the language and its implementation*. Addison-Wesley Longman Publishing Co., Inc., 1983.
- [26] J. Walker, D. Moon, D. Weinreb, and M. McMahon, "The Symbolics Genera Programming Environment," *IEEE Software*, vol. 4, no. 6, pp. 36–45, Nov. 1987.
- [27] A. A. diSessa and H. Abelson, "Boxer: A reconstructible computational medium," *Communications of the ACM*, vol. 29, no. 9, pp. 859–868, Sep. 1986.
- [28] C. M. Hancock, *Real-Time Programming and the Big Ideas of Computational Literacy*, Ph.D. dissertation, 2003.
- [29] S. L. Tanimoto, "A Perspective on the Evolution of Live Programming," in *Proceedings of the 1st International Workshop on Live Programming*, ser. LIVE '13, IEEE Press, San Francisco, California, 2013, pp. 31–34.
- [30] L. Andersen, M. Ballantyne, and M. Felleisen, "Adding interactive visual syntax to textual code," *Proceedings of the ACM on Programming Languages*, vol. 4, pp. 1–28, Nov. 2020.
- [31] C. Omar, D. Moon, A. Blinn, I. Voysey, N. Collins, and R. Chugh, "Filling typed holes with live GUIs," in *Proceedings of the 42nd ACM SIGPLAN International Conference on Programming Language Design and Implementation*, ACM, Jun. 2021.
- [32] M. Resnick, J. Maloney, A. Monroy-Hernández, et al., "Scratch," *Communications of the ACM*, vol. 52, no. 11, pp. 60–67, Nov. 2009.
- [33] I. Epic Games, *Blueprints Visual Scripting*. <https://docs.unrealengine.com/5.0/en-US/blueprints-visual-scripting-in-unreal-engine/>.
- [34] M. Kölling, N. C. C. Brown, and A. Altmir, "Frame-Based Editing," in *Proceedings of the Workshop in Primary and Secondary Computing Education*, ACM, Nov. 2015.
- [35] M. Read and C. Marlin, "Generating direct manipulation program editors within the MultiView programming environment," in *Joint proceedings of the second international software architecture workshop (ISAW-2) and international workshop on multiple perspectives in software development (Viewpoints '96) on SIGSOFT '96 workshops -*, ACM Press, 1996.

- [36] M. Boshernitsan, M. Downes, and M. Downes, "Visual Programming Languages : A Survey Marat Boshernitsan Michael Downes," 2004.
- [37] W. Citrin, M. Doherty, and B. G. Zorn, "The Design of a Completely Visual Object-Oriented Programming Language," 1994.
- [38] P. Cox, F. Giles, and T. Pietrzykowski, "Prograph: A step towards liberating programming from textual conditioning," in *[Proceedings] 1989 IEEE Workshop on Visual Languages*, IEEE Comput. Soc. Press.
- [39] M. M. Burnett and A. L. Ambler, "Interactive Visual Data Abstraction in a Declarative Visual Programming Language," *Journal of Visual Languages amp; Computing*, vol. 5, no. 1, pp. 29–60, Mar. 1994.
- [40] M. Najork and S. Kaplan, "The CUBE languages," in *Proceedings 1991 IEEE Workshop on Visual Languages*, IEEE Comput. Soc. Press.
- [41] M. Erwig and B. Meyer, "Heterogeneous visual languages-integrating visual and textual programming," in *Proceedings of Symposium on Visual Languages*, IEEE Comput. Soc. Press.
- [42] C. Omar, Y. S. Yoon, T. D. LaToza, and B. A. Myers, "Active code completion," in *2012 34th International Conference on Software Engineering (ICSE)*, IEEE, Jun. 2012.
- [43] JetBrains s.r.o., *Mps: The Domain-Specific Language Creator by JetBrains*, <https://www.jetbrains.com/mps/>, 2022.
- [44] I. Arawajo, A. DeArmas, M. Roberts, S. Basu, and T. Parikh, "Notational Programming for Notebook Environments: A Case Study with Quantum Circuits," in *The 35th Annual ACM Symposium on User Interface Software and Technology*, ACM, Oct. 2022.
- [45] A. Cypher, Ed., *Watch what I do: Programming by demonstration*. The MIT Press, 1994.
- [46] S. Kandel, A. Paepcke, J. Hellerstein, and J. Heer, "Wrangler," in *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems*, ACM, May 2011.
- [47] Z. Liu, J. Thompson, A. Wilson, et al., "Data Illustrator," in *Proceedings of the 2018 CHI Conference on Human Factors in Computing Systems*, ACM, Apr. 2018.
- [48] D. Ren, B. Lee, and M. Brehmer, "Charticulator: Interactive Construction of Bespoke Chart Layouts," *IEEE Transactions on Visualization and Computer Graphics*, vol. 25, no. 1, pp. 789–799, Jan. 2019.
- [49] A. Satyanarayan and J. Heer, "Lyra: An Interactive Visualization Design Environment," *Computer Graphics Forum*, vol. 33, no. 3, pp. 351–360, Jun. 2014.
- [50] B. Victor, "Drawing Dynamic Visualizations," <https://vimeo.com/66085662>, Presented at the Stanford HCI seminar on February 1, 2013, 2013.
- [51] K. S.-P. Chang, "A Spreadsheet Model for Using Web Services and Creating Data-Driven Applications," *Carnegie Mellon University*, 2016.
- [52] S. Oney, B. Myers, and J. Brandt, "Interstate," in *Proceedings of the 27th annual ACM symposium on User interface software and technology*, ACM, Oct. 2014.
- [53] T. Schachman and J. Horowitz, *Apparatus: A Hybrid Graphics Editor and Programming Environment for Creating Interactive Diagrams*. <http://aprt.us/>.
- [54] H. Xia, B. Araujo, T. Grossman, and D. Wigdor, "Object-Oriented Drawing," in *Proceedings of the 2016 CHI Conference on Human Factors in Computing Systems*, ACM, May 2016.
- [55] F. Dib, *Regex101: Build, Test, and Debug Regex*. <https://regex101.com/>.
- [56] gskiner, *RegExr: Learn, Build, Test RegEx*. <https://regexr.com/>.
- [57] S. Toarca and P. Prelich, *Debuggex: Online Visual Regex Tester. JavaScript, Python, and PCRE*. <https://www.debuggex.com/>.
- [58] A. M. McNutt, "No Grammar to Rule Them All: A Survey of JSON-style DSLs for Visualization," *IEEE Transactions on Visualization and Computer Graphics*, pp. 1–11, 2022.
- [59] M. B. Kery, D. Ren, F. Hohman, D. Moritz, K. Wongsuphasawat, and K. Patel, "Mage: Fluid Moves Between Code and Graphical Work in Computational Notebooks," in *Proceedings of the 33rd Annual ACM Symposium on User Interface Software and Technology*, ACM, Oct. 2020.
- [60] C. Omar, I. Voysey, R. Chugh, and M. A. Hammer, "Live functional programming with typed holes," *Proceedings of the ACM on Programming Languages*, vol. 3, pp. 1–32, Jan. 2019.

- [61] Observable Inc., *Quickly Explore Analyze Your Data With Data Table Cell / Observable / Observable*, <https://observablehq.com/@observablehq/introducing-data-table-cell>, 2022.
- [62] I. Hex Technologies, *Hex - Do More with Data, Together*. <https://hex.tech>.
- [63] K. Piersol, "A close-up of OpenDoc," *Byte*, vol. 19, no. 4, pp. 183–188, 1994.
- [64] Mozilla Development Network contributors, *Web Components / MDN*. https://developer.mozilla.org/en-US/docs/Web/Web_Components.
- [65] Meta Platforms, Inc., *React – A JavaScript library for building user interfaces*, <https://reactjs.org/>, 2022.
- [66] HASH, *Block Protocol*. <https://blockprotocol.org/>.